



SARUS AEROSPACE
PRIVATE LIMITED · INDIA

PROJECT REPORT

Sarus Operation Planner 1.0.0

What we built, how we built it, what is different from ArduPilot Mission Planner, how we tested it, and what it gives Sarus.



PRODUCT

Ground control station

VERSION

1.0.0 (Windows)

DATE

4 October 2026

STATUS

Release candidate

01 Summary

Sarus Operation Planner is the Windows program our pilots use to set up, plan, fly and review ArduPilot aircraft. We built it from ArduPilot Mission Planner 1.3.83 and left every screen, button, feature and keyboard shortcut where Mission Planner has it.

Four things changed. The program now carries the Sarus name, logo, colours and installer, and it sits next to an existing Mission Planner installation without touching it. Parameter editing no longer stands in the way of tuning: a value outside ArduPilot's recommended range turns amber instead of being refused, and after each write the program asks the aircraft what it actually kept. We also found a fault, inherited from Mission Planner, that let replies from the aircraft go missing when two requests were waiting at once. That is fixed. Finally, every change is now compiled, tested, packaged and test-installed by GitHub's cloud machines, with no manual steps.

Alongside the program, Sarus now has its own firmware line: an ArduPilot 4.7.1 fork that identifies itself as Sarus and is built automatically for seven flight-controller families.

0 changes to Mission Planner's shortcuts, screens, menus and controls	0 / 3,298 aircraft replies lost under stress after the fix (before: 13 of 105)	98 VTOL missions flown back to back in a 4-hour simulator run, no failures	~3,070 parameters written, read back and restored on QuadPlane, Copter and Rover
---	--	--	--

BEFORE REAL FLIGHTS

So far every flight has been in ArduPilot's official simulator. The Field Test Checklist, done with a real aircraft on the bench and then in a first flight, has to be completed before the program controls a Sarus aircraft.

02 The basics

Readers new to drone software may find this chapter useful. Abbreviations are explained in chapter 10.

Three parts of one system

An ArduPilot drone has three software-related parts. On the aircraft sits the **flight controller**, a small computer that reads gyroscopes, accelerometers, compass, GPS, barometer and airspeed hundreds of times a second and drives the motors and control surfaces. It runs the **firmware**, which decides how the aircraft flies: stabilisation, flight modes, missions, failsafes. Roughly 1,000 to 1,500 settings, called parameters, adjust that behaviour. On the ground, the **ground control station** shows what the aircraft is doing and lets the operator plan missions, change parameters, calibrate sensors, install firmware and download logs. Sarus Operation Planner is that ground station; ArduPilot, and now the Sarus fork of it, is the firmware; the flight controllers are off-the-shelf Pixhawk-class boards.



Figure 1. How Sarus Operation Planner fits into an ArduPilot drone system.

MAVLink

Aircraft and ground station talk in MAVLink, a compact message format. The aircraft streams its attitude, position and battery state several times a second. The ground station sends requests, such as "what is TECS_CLMB_MAX?" or "switch to AUTO", and waits for the matching answer. Radio links drop messages now and then, so the ground station has to notice a missing reply and ask again.

Mission Planner

Mission Planner is the free, open-source ground station written by Michael Osborne and the ArduPilot community. It is the most complete ArduPilot ground station for Windows and the one most ArduPilot pilots learn on. Its licence, the GNU GPL version 3, lets any company copy, change and sell it, on one condition: the changed source code must be published under the same licence.

QuadPlane

Our main airframe is a QuadPlane, a fixed-wing aircraft with four extra lift motors. It takes off and lands like a multirotor and cruises like an aeroplane, and ArduPilot Plane flies both phases. Speed and height in winged flight are handled by a controller called TECS. Being able to tune TECS and the VTOL (Q_) parameters without the

ground station getting in the way was one of the main requirements for this project.

03 Requirements and how they were met

REQUIREMENT	HOW IT WAS MET
Exactly Mission Planner's features	No button, screen, feature or shortcut was removed, moved or rebound. A scan of the source compares every menu, screen, control and shortcut with stock Mission Planner; it finds no differences apart from three intended name changes.
Sarus look	Sarus Dark (default) and Sarus Light themes, redrawn menu icons, the crane logo in the top-right corner (it opens sarusaerospace.in), splash screen, program icon and installer. HUD and map colours stay as they were, so pilots read them the same way.
New firmware visible at once	Install Firmware reads ArduPilot's live firmware list, so a new ArduPilot release shows up the day it is published. The Sarus firmware list is read the same way, and Sarus builds appear next to ArduPilot's.
Tune freely, still see errors	Out-of-range values are accepted and marked amber. Read-only parameters can be changed after a confirmation. Every written value is checked against what the aircraft stored.
One setup file for any Windows 10 or 11 PC	A standard Windows installer, built in the cloud and installed as a test on fresh Windows Server 2022 and 2025 machines, which correspond to Windows 10 and 11.
No defects	Simulator flights for three vehicle types, parameter sweeps, every setup page, shortcuts, bad-link flights, a 4-hour endurance run and stress tests (chapter 6). Faults found in Mission Planner itself were fixed and are listed in chapter 5.

04 How the work was done



Figure 2. The path from source code to a released installer.

A stable starting point

We took Mission Planner's last stable release, 1.3.83, rather than its development branch, which did not compile at the time. Before changing anything we built and tested the stock program to have something to compare against. It passed 3 of 3 simulator flights. Seven of its 38 unit tests failed, and they fail the same way in Mission Planner itself.

Small changes, one at a time

The work went in as 18 separate commits, each with a description, so any one of them can be reviewed or undone without touching the others. Where Mission Planner already offers a way to customise something (theme files, menu icon overrides, the logo file, the application name), we used it instead of editing program logic. Name replacements in the source were made by script, and the script stops if the number of replacements differs from what was expected.

A test pilot made of software

Most of the testing was done by a test harness we wrote for this project. It starts the real Sarus program, connects it to ArduPilot's simulator and then works the program the way a pilot would: connecting, opening every screen, changing parameters, uploading missions, arming, flying and pressing shortcuts, checking the result each time. Each test was run against stock Mission Planner first, so every Sarus result has a Mission Planner result next to it.

Builds in the cloud

The source code is public at github.com/LikhitS/SarusOperationPlanner, as the GPL requires. Each change pushed there is compiled on a GitHub Windows machine, unit-tested and packaged as an installer. Two more fresh Windows machines then install it silently next to a stock Mission Planner, start it, leave it running for a minute and uninstall it. Pushing a version tag such as v1.0.0 publishes the result as a GitHub release, which is where both the download page and the program's own updater look for new versions.

05 What changed

5.1 Identity and installation

AREA	MISSION PLANNER	SARUS OPERATION PLANNER
Name and logo	Mission Planner, ArduPilot logo	Sarus Operation Planner, crane logo; clicking it opens sarusaerospace.in
Look	Mission Planner themes	Sarus Dark (default) and Sarus Light, new menu icons from Tabler Icons (MIT licence); Mission Planner's themes can still be chosen
Installer	Mission Planner MSI	Its own MSI with its own product identity, so it installs beside Mission Planner rather than over it; its own file associations for .tlog, .bin and .log
Settings and logs	Documents\Mission Planner	Documents\Sarus Operation Planner and ProgramData\Sarus Operation Planner
Program updates	ArduPilot servers	Sarus releases on GitHub
Usage statistics	On by default	Off by default (CONFIG → Planner can turn them on)
Firmware list	ArduPilot	ArduPilot plus Sarus builds
Version	1.3.83	1.0.0; the file version stays 1.3.83 to show the Mission Planner base

USB drivers and their signing certificate, map providers, the parameter documentation source and every feature and shortcut were deliberately left as Mission Planner has them.

5.2 Parameter editing

In Mission Planner, typing a value outside the recommended range brings up a Yes/No question for every such value, read-only parameters cannot be changed at all, and the table shows whatever was typed even if the aircraft rounded or limited it. Sarus handles all three differently. Out-of-range values are accepted without a question; the cell turns amber and its tooltip gives the recommended range. A read-only parameter can be changed after a clear warning. And once a write is done, the program reads every written value back from the aircraft. Anything the aircraft did not keep is listed in a "Values changed by aircraft" window, and the table shows the real value.

FOUND DURING TESTING

Asked to set Q_LOIT_ACC_MAX_M to 9.81, the aircraft confirmed it and then ran at 5.66. Mission Planner would have shown 9.81. Sarus shows 5.66 and says so.

5.3 Lost replies

When Mission Planner waits for an answer from the aircraft, the waiting code reads incoming messages one by one and throws away whatever it is not waiting for. That works until two parts of the program wait at the same moment, say a background camera query while the pilot reads parameters. Then one of them can pick up the

other's reply and discard it. The aircraft answered; the pilot still gets "timeout" or "command failed".

In Sarus, a request first registers for the reply it expects and only then sends the question. Every waiting request gets its own copy of each incoming message, so nothing can be taken from it. This now covers parameter reads and writes, command acknowledgements, commands with coordinates, mission upload and download, setting the current waypoint, and the flight-log list. The numbers below come from the simulator under deliberately heavy load.

OPERATION	MISSION PLANNER	SARUS 1.0.0
Parameter reads	6 of 45 lost	0 of 600 lost
Command acknowledgements (arm, mode)	5 of 15 lost	0 of 1,811 lost
Mission upload and download	4 of 10 failed	0 failed
Commands with coordinates	2 of 45 lost	0 of 887 lost
Flight-log list during parameter reads	not measured	0 of 326 failed

5.4 Other Mission Planner faults we fixed

The update check showed an error whenever the update server had no file yet; it now reports quietly that there is no update. Plugins compiled by an older version were reused after an upgrade and then failed every second, so the plugin cache now includes the program's build identity. The updater used to download a whole release file just to learn its size, where reading the size header is enough. When a legacy rally-point request was lost, the retry went out as a fence request and could never succeed; it now sends the right message.

The silent-install problem took longest to find. Mission Planner's installer starts a USB driver wizard that waits for a click. In a silent installation (`msiexec /qn`, as IT departments use) nobody can click, and the installation never ends. Stock Mission Planner hung there on both of our test machines. Sarus runs that driver step quietly when the installation is silent; normal installations are unchanged. In a silent install Windows adds only drivers whose publisher is already trusted, which keeps the same trust model as Mission Planner.

06 Testing

TEST	WHAT WAS DONE	RESULT
Feature parity	Scan of every menu, screen, control and keyboard shortcut, compared with stock Mission Planner	No differences besides 3 intended renames
Unit tests	Mission Planner's own test suite	31 of 38, same as stock (7 known upstream failures)
QuadPlane flights	Connect, upload mission, arm, VTOL take-off, waypoints, VTOL landing; 42 checks per flight	3 of 3 flights, 42 of 42 checks
Copter flight	Take-off, waypoints, return to launch	Pass
Rover drive	Parameters, modes, a square mission and return; parameter editor checks; log list	59 of 59 checks
Endurance	QuadPlane VTOL missions back to back for 4 hours	98 of 98, memory +24 MB
Parameter sweeps	Every vehicle parameter written, read back and restored	~1,180 QuadPlane, ~1,000 Copter, 886 Rover
Setup pages	Every SETUP and CONFIG page opened while connected	46 QuadPlane, 48 Copter
Keyboard shortcuts	F2, F3, F4, F5, F12 and Ctrl+F, P, G, X, L, W, J, Y	All pass
Bad radio link	5% packet loss, 300 ms delay and a 10-second blackout in flight	Telemetry back 0.3 s after the link returned
Stress	Many requests at once (section 5.3)	No lost replies
Clean-machine install	Silent install beside Mission Planner on fresh Windows Server 2022 and 2025, run for 60 s, uninstall	Pass on both, 228–236 MB in use after 60 s

The parameter sweeps leave out the simulator's own physics settings (SIM_*) and the identity and link settings whose change would cut the test's connection.

What the tests showed about ArduPilot

The sweeps also exposed firmware behaviour that every ground station inherits, and which Sarus now reports to the pilot. Q_LOIT_ACC_MAX_M and LOIT_ACC_MAX_M are limited after the aircraft has accepted them. EK3_BCN_DELAY, EK3_FLOW_DELAY and LAND_THR_SLEW are stored in 8 bits and stop at 127, although their documentation allows more. The firmware overwrites BARO*_GND_PRESS by itself, and MIS_TOTAL cannot be written. In the simulator, parameter storage fills up after about 600 changed values. These are the first candidates for changes in the Sarus firmware.

07 What it gives Sarus

Pilots lose nothing. The screens and shortcuts are the ones every ArduPilot pilot already knows, so existing Mission Planner tutorials still apply. What they gain is fewer false "timeout" and "command failed" messages, which matters most on long-range radio links, a parameter table that shows the values the aircraft is actually using, and room to tune the QuadPlane without being stopped by warnings.

For the company the gain is control. Sarus can ship a ground station under its own name with its aircraft, choose when to release, and fix problems without waiting for another project's schedule. The build and release pipeline is repeatable and tested, so a release does not depend on one person's laptop. It is also the base for the Sarus firmware and for the Linux, macOS and Android versions that come later.

08 Open items

ITEM	STATUS AND NEXT STEP	OWNER
Real-hardware test	Bench and first flight with a Pixhawk QuadPlane, following the Field Test Checklist. Required before operational use.	Flight team
Code signing	Until the installer is signed, Windows warns "Windows protected your PC". Needs a D-U-N-S number, then an OV certificate with cloud signing; the credentials go into GitHub Secrets.	Management
First app release	Push the tag v1.0.0 once the field test has passed.	Management, engineering
First firmware release	Done: sarus-v4.7.1-S1 was released on 4 October 2026 after passing the simulator tests. Next is the same field test on Sarus firmware.	Engineering
Legal	Lawyer review of terms of use, privacy notice and safety disclaimer; trademark search; DGCA obligations under the Drone Rules 2021.	Management
High-resolution screens	Text is slightly soft above 100% Windows scaling, as in Mission Planner.	Engineering
Other platforms	Linux, macOS and Android, reusing the same core code. See the Future Plans document.	Engineering

09 Licence and credits

Sarus Operation Planner is free software under the GNU General Public License version 3. The complete source, Sarus changes included, is at github.com/LikhitS/SarusOperationPlanner, and the Sarus firmware source at github.com/LikhitS/ardupilot. As the licence states, there is no warranty.

The program is based on Mission Planner by Michael Osborne and the ArduPilot contributors. ArduPilot and Mission Planner belong to their respective owners, who do not endorse this product. The menu icons come from Tabler Icons, under the MIT licence.

10 Abbreviations

TERM	MEANING
ACK	Acknowledgement, the aircraft's reply confirming a command
ADS-B	Automatic Dependent Surveillance–Broadcast, position broadcasts used for traffic awareness
AHRS	Attitude and Heading Reference System
CI	Continuous Integration: every change is built and tested automatically (here by GitHub Actions)
DGCA	Directorate General of Civil Aviation, India
DPI	Dots per inch; the Windows screen-scaling setting
D-U-N-S	Data Universal Numbering System, a company identifier that code-signing authorities require
EKF	Extended Kalman Filter, ArduPilot's position and attitude estimator (EK3 is version 3)
ESC	Electronic Speed Controller, which drives a brushless motor
FBWA / FBWB	Fly-By-Wire A and B, assisted fixed-wing modes
GCS	Ground Control Station
GPL	GNU General Public License
GPS / GNSS	Global Positioning System / Global Navigation Satellite System
HDOP	Horizontal Dilution of Precision; lower means a better GPS fix
HUD	Head-Up Display, the artificial horizon and instruments on the DATA screen
MAVLink	Micro Air Vehicle Link, the message format between aircraft and ground
MP	Mission Planner
MSI	Microsoft Installer, the Windows setup file format
OV	Organisation Validated, a type of code-signing certificate
PID	Proportional–Integral–Derivative, the control loops tuned to make an aircraft fly well
QHOVER / QLOITER / QRTL	QuadPlane VTOL modes: hover, hold position, return to launch
RC	Radio Control: the pilot's transmitter and the receiver on the aircraft
RTL	Return To Launch
SITL	Software In The Loop, ArduPilot's simulator, which runs the real firmware on a PC
TECS	Total Energy Control System, ArduPlane's combined speed and height controller
tlog / bin	Telemetry log recorded by the ground station / log recorded on the flight controller
UI / UX	User Interface / User Experience
VTOL	Vertical Take-Off and Landing
WP	Waypoint

Prepared by: Engineering

Reviewed by:

Approved by: